

Polypress

A lossless compressor for data tables. Measured against production binaries on real public data — not simulations, not estimates.

1.49× smaller than Parquet	73.7× compression, 278 MB file	174 MB/s decode throughput	exact round-trip fidelity
--------------------------------------	--	--------------------------------------	-------------------------------------

What it is

Polypress compresses tabular data — CSV, TSV, JSON, Parquet — and reproduces the logical table exactly: every column name, the column order, the row order, and every cell as the identical string. Every compression is verified in memory before anything is written to disk.

It beats every general-purpose compressor tested (xz, zstd, brotli, bzip2, gzip), the specialised numeric codecs shipped in ClickHouse (Gorilla, DoubleDelta, T64, Delta), the HPC array codecs zfp and fpzip, and Apache Parquet in four codecs — on every dataset tested.

The headline result

NHAMCS Adult, a CDC National Hospital Ambulatory Medical Care Survey extract: 96,539 rows × 209 columns, 277.9 MB. The full file, not a sample. Parquet reproduced the printed text of all 209 columns exactly here, so none of its size advantage comes from discarding formatting — this is a like-for-like comparison.

Codec	Bytes	Ratio	Time
Polypress	3,956,941	73.65×	25 s
parquet + brotli	5,881,047	49.55×	25 s
parquet + zstd	5,984,000	48.70×	21 s
zstd --ultra -22	6,178,417	47.17×	90 s
xz -9e	6,432,356	45.31×	28 s
brotli -q 11	7,132,152	40.86×	141 s
parquet + gzip	7,854,508	37.10×	1 s
bzip2 -9	8,989,156	32.42×	33 s
parquet + snappy	16,445,675	17.72×	0 s
gzip -9	18,781,409	15.52×	3 s

Parquet is the honest competitor: nobody stores a 209-column survey as compressed CSV. Polypress is **1.49× smaller than the best Parquet configuration**, at the same encode time.

Is that the modelling, or just a better final compressor?

A fair objection, and one worth answering before it is asked. Parquet **cannot use xz at all** — its options are snappy, gzip, brotli, zstd and lz4, and asking pyarrow for xz returns an unsupported-codec error. So some of the

margin above could be the finisher rather than the front end.

It is not. Re-finishing Polypress with the *same* codec Parquet is using, across 18 tables, Polypress wins **18 of 18 at zstd-22 and 17 of 18 at brotli-11**, by margins from 1.06× to 5.6×. The single loss is an adversarial table, by 11%. Strip xz out entirely and the gap barely moves.

Dataset	ppz+zstd	pq+zstd	ppz+brotli	pq+brotli
CDC notifiable disease	41,531	191,910	37,027	223,929
NOAA climate, SEA	6,896	36,732	6,392	35,524
Seattle fire 911	612,622	1,138,757	576,554	1,095,103
Chicago permits	918,844	1,353,332	864,525	1,312,856
WA EV population	272,923	528,746	254,441	502,133

The same measurement prices the speed trade, since it is the same swap: Polypress finished with zstd instead of xz is 8% larger and several times faster, and still beats Parquet on every table.

Why it wins

Three ideas. The third is the one nothing else does.

1. Local function building

Fit a low-degree polynomial to the last few values in a column, extrapolate one step, store only the error. Because the fit is re-centred at every cell, the coefficients never have to be stored — the decoder already holds the neighbours. Order- k extrapolation turns out to be exactly the k -th finite difference.

2. The same idea in two dimensions

Where adjacent numeric columns are *commensurable* — same decimal places, same magnitude — a cell is predicted from its left, upper and upper-left neighbours. Detection matters as much as the predictor: differencing “Model Year” against “Make” is meaningless, so groups form only where columns are genuinely comparable.

3. Cross-column structure by reordering

Table columns are not independent — City is nearly determined by Postal Code. Rather than model that, Polypress **sorts the rows by the parent column**: equal parents become adjacent, the child collapses into long runs, and the entropy coder eats it. **The permutation costs nothing to store**, because the decoder has already rebuilt the parent and recomputes the same stable sort. Parents are chosen by conditional entropy with a Miller-Madow correction and arranged into a tree.

On the survey file, **172 of 200 dictionary columns were sorted by a parent**. Survey answers predict each other heavily, and no columnar format exploits this — Parquet compresses each column chunk independently by design. That is the entire margin.

Measured directly: on 40,000 rows of vehicle-registration data, coding City given Postal Code took 2,328 bytes by reordering versus 6,901 bytes using an adaptive context-modelling range coder — and reordering is far faster, because the work moves into C instead of a Python loop.

Thirteen real datasets, fetched and measured end to end

Six curated datasets was never a claim. These are pulled from government open-data portals by one command, chosen across *shapes* rather than subjects, since shape is what decides whether this codec wins. All verified exact round-trip.

Dataset	Shape	Win vs best other
CDC notifiable disease	150,000 × 16	3.70× pq+brotli
Seattle fire 911	200,000 × 7	1.77× xz -9e
WA EV population	200,000 × 16	1.75× xz -9e
Austin 311	150,000 × 19	1.61× xz -9e
NYC collisions	150,000 × 29	1.44× xz -9e
Chicago crimes	150,000 × 22	1.44× xz -9e
NYC 311	60,000 × 44	1.31× xz -9e
NYC baby names	29,685 × 6	1.26× pq+brotli
USGS earthquakes 2023	16,190 × 22	1.25× bzip2 -9
USGS earthquakes 21-22	16,707 × 22	1.21× bzip2 -9
Chicago permits	80,000 × 116	1.12× xz -9e
NOAA climate, SEA	79 × 106	1.11× bzip2 -9
NOAA climate, ORD	69 × 102	1.11× brotli -q 11

13 of 13 wins, but read the spread, not the headline. Median 1.31×. Only six clear 1.4×. Chicago permits is the informative one: 116 columns, 106 dictionary-encoded, 80 successfully sorted by a parent — the machinery firing about as hard as it can — and still only 1.12×, because eight free-text columns hold most of the bytes. **Width is not the predictor; the fraction of the file that is modellable is.** Parquet failed the exact-printed-text check on 8 of the 13, so its column is flattered on most rows.

Matrix-shaped tables

Every dataset above is survey, administrative, incident or text-heavy data — none is the shape the planar predictor was built for, so for a long time that predictor's claim rested on data no benchmark touched. These three fix that, and they are fetched by one command like the rest.

Dataset	Shape	Polypress	Best other	Win
Treasury yield curve 1990–2025	9,006 × 9	34,891	64,836 xz	1.86×
Weather, 10 sensors hourly × 10y	87,672 × 11	503,460	1,078,640 xz	2.14×
Hourly temperature, 24 cities	26,304 × 25	414,471	687,390 bzip2	1.66×

Getting this data in immediately exposed a defect worth more than the benchmark. The yield curve classified as *zero* numeric columns, so no group could form — because four cells out of 72,048 are blank, and the numeric test was all or nothing. **Four blank cells were costing 41% of that file.** Numeric columns now carry exceptions: unrepresentable cells are stored separately and their slots filled in, which keeps every column the same length and so keeps the planar predictor able to stack them.

Against specialised numeric codecs

General-purpose tools were never the real competition for numeric tables. These are the shipped implementations — ClickHouse 26.8.1, and zfp/fpzip from the HPC world — on the Treasury yield curve (7,003 × 8). Note this is an *earlier, shorter* extract than the 9,006 × 9 file in the table above, so the byte counts are not comparable between the two tables — only within each one.

Codec	Bytes	Ratio
Polypress	21,981	13.07×
ClickHouse Delta + ZSTD(22)	43,375	6.62×
zfp int32 lossless + xz	45,776	6.27×
ClickHouse DoubleDelta + ZSTD(22)	49,647	5.78×
ClickHouse T64 + ZSTD(22)	55,288	5.19×
ClickHouse Gorilla + ZSTD(22)	141,642	2.03×
fpzip lossless (float64)	255,606	1.12×

Every codec above predicts *down* a column. None predict *across* columns. Note the fair-comparison caveat: Gorilla, zfp and fpzip assume genuinely continuous floating-point physics data, so their float64 entries look poor on decimal-quantised financial values — the int32 rows are the fair ones, and Polypress still leads by 1.96×

Speed

Encode beats every max-level general compressor. Decode is roughly comparable. Nothing here approaches the fast tier — zstd -3 encodes at 173 MB/s and always will — and this is still Python plus numpy with a small C library for the hot loops.

Dataset	Encode MB/s	Decode MB/s
NHAMCS survey	11.9	174
CDC notifiable disease	32.0	166
Weather, 10 sensors hourly	2.8	57
Treasury yield curve	1.6	55
— for reference: xz -9e	3.1	108
— for reference: brotli -q 11	1.0	255
— for reference: zstd -3	130	440

A 278 MB file compresses in 25 seconds and restores in 1.4 seconds. **The honest caveat is that never-worse costs encode time and the bill has grown.** Every guarantee in this codec works by encoding the table both ways and keeping the smaller, so a table eligible for a guard is encoded twice. The 2026-07-29 changes made encode 1.86× slower for 1.66% smaller output; a cheap screen recovered part of that, but three large datasets still encode twice and keep the first result, because their columns are cheaper in isolation and lose only once the whole file is assembled. Narrow tables and wide ones with nothing to recover are untouched.

Files larger than memory

The single-shot codec holds the whole table as Python strings — a measured 8.1× expansion — so a 50 GB input would need hundreds of gigabytes and would not run. Polypress therefore also ships a block-streaming

mode: the table is split into row blocks, each compressed independently behind an index, with peak memory set by the caller. The trade is explicit, because cross-column reordering only sees correlations inside a block.

Memory budget	Blocks	Ratio	Peak RSS
single-shot	1	73.65×	2.36 GB
2.0 GB	3	72.26×	1.89 GB
1.0 GB	6	69.12×	1.06 GB
0.25 GB	10	64.22×	0.64 GB

Blocks are independent, so this is also what makes parallelism possible. Projected for a 50 GB file at measured single-core throughput: about 95 minutes to compress, about 4 minutes to restore, with memory bounded at the chosen budget. That projection is arithmetic, not a measurement.

Polypress On-Demand — column random access

Parquet exists to read one column out of 209 and touch nothing else. The archival codec cannot, because its size win comes from *coupling* columns. That looked fatal for random access, and is not: the parent tree is shallow. On the 209-column survey the ancestor chain averages 2.78 and never exceeds 5, so reading one column costs about four column decodes rather than 209 — and each hop needs only an ancestor's ordering, never its string table.

Metric	Polypress On-Demand	Parquet
Archive size	4.2 MB (66.9×)	5.6 MB (49.6×)
Single-column read, 5 columns	0.061 s	0.055 s
Bytes touched per column	0.2–0.4 MB	—

1.33× smaller than Parquet, at within about 10% of its read speed, on the access pattern Parquet was designed for. This is a separate build from the archival codec and uses zstd rather than xz, because on-demand reads are dominated by decompression.

What is honestly not done

Stated plainly, because these decide whether this is a research result or a product.

- **One specimen at the top end.** The 73× result is a single survey file and may be a property of that file. The thirteen-dataset corpus since added establishes the *spread* — median 1.31×, worst 1.11× — but not that the top end generalises. That still needs three or four more wide categorical tables: NHAMCS ED, NAMCS, BRFSS.
- **Encode is slow, and deliberately so.** 1.6–32 MB/s depending on the table, against zstd -3 at 173 MB/s. Part is Python; part is that every never-worse guarantee is implemented by encoding the table both ways and keeping the smaller. The parent search is also quadratic in the column count, which is what drags the widest tables down.
- **Single-threaded.** Block independence makes parallelism easy, but it is not implemented.
- **No format specification or versioning policy.** Nobody should license a format they cannot independently implement. Fuzzing does now exist — random adversarial tables through both implementations, corrupt

archives under a hard memory cap, and headers that are well-formed and deliberately dishonest.

- **The reordering trick is prior art, and an earlier version of this write-up claimed otherwise.** US 8,312,026 B2 (Kiem-Phong Vo, AT&T, filed 2009, granted 2012, now expired) discloses it in full: per-column orderings derived from a predictor field's stable argsort, a dependency tree so the predictor is always inverted first, the permutation never stored, and the predictor chosen by measured compressed size. Earlier still, Vo & Vo, DCC 2004. This project reached the same design independently and without knowledge of it; that is evidence the design is right, not evidence of priority, and the novelty claim is withdrawn. The patents are expired, so there is no restriction on using the code.
- **The predictors are prior art too.** The planar predictor is Lorenzo (Ibarria, Lindstrom, Rossignac & Szymczak, 2003, used in fpzip and SZ); MED is JPEG-LS; the range coder is LZMA's. The commensurable-group detection is the one part of the front end no anticipating disclosure was found for, and it is most likely obvious in combination.
- **Nobody outside the project has run it.** The macOS build is unsigned, so Gatekeeper reports it as damaged until someone pays for a developer certificate.

How the results were produced

Contenders are real binaries and real libraries, invoked at their strongest settings: xz -9e, zstd --ultra -22, brotli -q 11, bzip2 -9, gzip -9, pyarrow Parquet with zstd-22 / brotli-11 / gzip-9 / snappy, the ClickHouse 26.8.1 binary for Gorilla, DoubleDelta, T64 and Delta, and zfp / fpzip from PyPI.

Two checks guard every number. First, fidelity is verified on both sides: Polypress round-trips are compared cell by cell, and Parquet was checked to confirm it was not winning size by silently discarding formatting. Second, an earlier version of this project claimed a 1% win over xz that turned out to be CSV quote-stripping rather than compression — feeding xz the same canonicalised bytes matched it to within 68 bytes. That claim was retracted, and the same control is now run before any comparison is reported.

Correctness is covered by 41 fidelity cases run twice, once through the C accelerator and once through the numpy fallback, on the principle that an accelerator which disagrees with the reference is not an accelerator but a second codec. Those tests caught three real defects the benchmarks never would have: a 32-bit variant tail that truncated values past 2^{31} , newline-joined text storage that split any cell containing a newline, and a division by zero on an empty table.

There is also a second, independent implementation. The standalone C binary reads and writes archives with no Python and no numpy — the point being that a researcher sent an archive should not have to build an environment to open it — and it is **byte-identical** to the Python encoder, not merely equivalent. That is verified on 48 fidelity cases including the container choice, plus 500 randomly generated adversarial tables, with zero differing bytes. Any divergence is a bug with a known location, and the Python implementation stays usable as the oracle. Reaching it required reproducing numpy's pairwise summation exactly, because a different last bit flips a comparison, picks a different parent, and changes every byte after it.

The decoder is treated as reading hostile input, because it reads files other people made. Three suites cover that: corrupt archives opened in a subprocess under a hard memory cap, random adversarial tables through both implementations, and — added after three unchecked array indices survived every mutation-based pass and had to be found by reading — archives whose header is well-formed and deliberately dishonest, claiming more columns than it carries, parents that do not exist, or row counts larger than the file. That last suite found a segfault on its first run.

Where this could be worth something

Cold archival of wide categorical tables — survey archives, regulatory retention, historical warehouse snapshots. Written once, read rarely, read whole. Storage is cheap, so a 1.5× improvement on a small warehouse is not a business; the case is strongest where data volumes are large, retention is mandatory, and the tables are wide and categorical. The On-Demand fork widens that to workloads that need column access, which is most analytics.

Datasets: US city and state open-data portals (NYC, Chicago, Seattle, Austin, Washington State), CDC (notifiable disease surveillance, NHAMCS, NHANES), USGS, NOAA, the US Treasury, and ERA5 reanalysis via Open-Meteo. Every one is fetched by a script in the repository with no credentials and no manual step, so every table in this document can be reproduced from a clean checkout. Benchmarks run on Apple silicon, macOS, Python 3.9. Every figure here is a measurement taken on that machine unless explicitly labelled a projection.